

The 57 checks, and how the coding assistant reaches Amazon

A companion to "How Sky Score works, in plain English" · 2026-09-21

In one breath: before any change to Sky Score is saved, 57 automatic checks run; 45 of them can refuse the change. Most of the code was written with a coding assistant, which reaches Amazon only through a pass that is allowed to touch Sky Score's own things and nothing else. The checks are what make that arrangement safe; the pass is what makes it limited.

- 1 What "preflight" is, and what happens when a check fails
- 2 All 57 checks, one line each
- 3 The rule that makes the checks worth having
- 4 How the coding assistant reaches Amazon
- 5 Five layers of guardrail
- 6 What a deploy looks like, step by step
- 7 Two days the guardrails said no

1 - What "preflight" is

Preflight is a checklist that runs on my computer before any change to the project is saved into its history. One command runs all 57 in sequence and prints one line per check: **PASS**, or **FAIL** with the reason.

- **45 are blocking.** If one fails, the change cannot be saved until it is fixed. There is no "save anyway" button; the rule in the project's own notes says a failing check must be fixed, not skipped, even if I ask.
- **12 are advisory.** They print a warning and let the change through. They are advisory for a reason each: some compare the live website with the working copy (which differ by design until the next deploy), and some need large data files that a fresh copy of the project would not have.
- **It takes about 24 minutes** when the flood-map cache is cold, about 15 when warm. Most of that is a robot driving the real website in a real browser at eleven screen sizes, and the checks that phone a government service to compare a number.
- **Six stages need the internet.** They can be skipped for an offline run, but a skipped stage still prints its own line saying SKIPPED, and the final verdict says "PASS (INCOMPLETE)". A stage that vanishes from a report is indistinguishable from one that passed; that was a real bug once.

2 - All 57 checks, one line each

Names as the report prints them; explanations in plain words. Advisory checks are in italics.

#	NAME IN THE REPORT	WHAT IT ASKS
Is the code tidy? (four checks, three languages)		
1	ESLint (8 targets)	Reads the JavaScript for mistakes a browser would not report: an unused name, a misspelt variable, a call to something that does not exist.
2	html-validate (9 pages)	Checks the nine public pages are well-formed HTML: every tag closed, every image labelled.
3	ruff (backend/lambda)	The same for the Python that runs on Amazon.
4	ruff (scripts, tests)	And for the scripts that build the data and the tests themselves. The tests were outside this for a month once; the one folder guarding the score engine was the one nothing checked.
Does the arithmetic still work? (unit tests)		
5	pytest (backend)	Hundreds of small tests of the scoring engine and the assistant: "given these inputs, the score must be 5.4"; "an invented number must be caught". Includes the ten tests of the forty-line check.
6	pytest (root)	Tests of the data scripts and the deploy tooling, including one that reads the whole deploy file (the Makefile) to make sure the runner can still understand it.
Do the two halves and the documents agree with the engine? (contracts)		
7	API base-URL drift (I-N5)	Every page that talks to the API uses the same address, held in one file. A page with its own copy of the address is a page that breaks silently when the address changes.
8	score sanity (live API)	Asks the live API for 28 postcodes, one per city, and checks each resolves to the right city with a well-formed score. The only check that can catch a data fault in the live system.
9	demo key reaches only /v1/score	Asks the running API whether the free and demo passes are refused on the paid batch route. The configuration says they should be; only the running system can prove it.
16	openapi == score engine	The published description of the API (which customers generate code from) must match what the engine actually returns: every field, every city, every buyer profile.
17	worked example reproduces	The methodology page shows a worked example for SW11 1AA. This recomputes it from the engine, input by input, weight by weight. It once passed for weeks while never comparing affordability at all.
Do the published numbers still equal their source? (the unusual ones)		

#	NAME IN THE REPORT	WHAT IT ASKS
10	prices == HM Land Registry	Every borough's price and growth figure, re-read from the Land Registry's file and compared. Written after a month where prices matched one edition and growth matched none.
11	aircraft bands == geometry	Every borough's aircraft-noise band, re-derived from the airport positions and the measured noise footprints.
12	crime == ONS Table C4	Every borough's crime rate against the statistics office's own table, with a per-city minimum so it cannot pass by comparing nothing.
13	flood == EA service (georef)	Asks the Environment Agency's own map service what risk it reports at sample points, and compares. Samples the edges of the map first, because a positioning error shows at the edges. The first version passed a known-broken map 9 times out of 9 until it did.
14	site == Lambda (91 boroughs)	Drives the real website, reads the score it draws for every borough, and compares with the API's answer. The check from the main guide's Part 5.
15	area names == MSOA names	Every neighbourhood label (281 of them) is checked against the official small-area names for that district. It caught six labels that had been wrong for months.
48	p8 == DfE KS4 2023/24	<i>Every borough's school figure against the Department for Education's file. Advisory only because that file cannot be downloaded automatically.</i>
49	borough bands == sources	<i>Road noise, air quality, flood, transport and healthcare shares, re-derived from the maps and registers. Advisory because the maps are large files a fresh copy would not have.</i>
50	neighbourhood medians == PPD	<i>The 481 neighbourhood prices, re-derived from the Land Registry's full sales file (155 MB, not kept in the project).</i>
51	aircraft footprint == DEFRA	<i>The measured size of each airport's noise footprint, re-derived from the noise maps.</i>
52	quiet estimate == DEFRA	<i>Measures how far the distance-based aircraft estimate is from the government's readings, over 35,352 London postcodes. The accuracy figure the API page publishes comes from here, so it is re-measured rather than quoted.</i>

Does the website behave, on every screen? (a robot drives it)

18	no em dashes (all pages)	A house style rule, enforced by a check rather than by memory.
20	self-hosted fonts (9 pages)	Every page loads its typefaces from our own site, not from Google, so no page depends on a third party or leaks a visit to one.
21	extension extraction	The browser extension (a demo for property listings) still reads the address, price and coordinates from a real saved listing page.
22	extension panel per env state	The extension's panel renders correctly for each shape of answer the environment endpoint can give, including "measured as quieter than 40 dB".
23	Playwright e2e (--workers=2)	A robot opens the live site and walks the main journeys: search, panels, saved places, install prompt.

#	NAME IN THE REPORT	WHAT IT ASKS
24	extension e2e	The same robot loads the extension into a real browser on a real listing and checks what it shows against the live API.
25	local smoke + registry	Opens the working copy of the site and checks it boots, every city has every registry field, and the map library is fetched exactly once.
26	degraded + offline fallbacks	Stalls the network, launches offline, and breaks the transport service on purpose, checking the site says the honest thing each time rather than a confident absence.
27	locator inset	The small UK map in the corner draws the right city markers and none of them is a keyboard trap.
28	selector tiers do not overlap	The country and city chips never overlap each other or fall off the strip, at every width.
29	every city switches	Clicks every city chip, on a phone and on a desktop, and counts the borough outlines drawn against that city's own file. Written the day six of nine cities were found dead on the live site.
30	legal links reachable on a phone	Privacy and terms can be reached from a phone. For ten days once, the mobile home page had exactly one visible link.
31	PWA installable (manifest + SW)	The site can still be installed as an app: the manifest and the offline worker are valid and every file the worker pre-caches exists.
32	changes page explains why	The page listing score changes between data editions explains each movement rather than just listing it.
33	map fits its box	At nine screen sizes including landscape phones, every city's map is drawn inside its own frame. 53 of 90 combinations were failing when this was written, with every other check green.
34	web/native layout split	The website and the phone-app build render their different layouts as intended; neither has silently become the other.
35	UK cities get UK panel content	The panel for a UK city shows UK data sources and stations, never the New York wording.
36	area pages carry real data	The 100 static borough pages each have real figures, unique titles, and no two share a block of text. A hundred near-identical pages would count against the site.
37	area pages match the live API	Those pages bake their scores in, so this compares all 99 against the live API in one request.
38	layers paint only real data	The map's colour layers (road noise, flood, air) paint a borough only where there is a reading, and the legend shows only bands that are actually painted. Once, seven cities were painted a confident colour meaning nothing.
39	panel says what it measured	Renders the borough panel and checks its caveat names the inputs it actually has. Nothing had rendered that panel before; it had shipped the word "undefined" to users.
40	favourites: keyboard + not nested	Saved places can be added and removed by keyboard, and no button sits inside another button.

#	NAME IN THE REPORT	WHAT IT ASKS
41	ranking claims only real order	When boroughs' scores are too close to rank meaningfully, the ranking table says so instead of implying an order.
42	panel contrast, borough (AA)	Measures the colour contrast of every visible piece of text in the borough panel against the accessibility standard, itself, rather than trusting a scanner that reports "could not determine" as a pass. Found 46 failures on its first run.
43	panel contrast, area (AA)	The same for the neighbourhood panel, including the state where the NHS helper says nothing is nearby.
44	responsive, source	At twelve screen sizes, nothing scrolls sideways, no control is pushed off-screen, covered by something else, or clipped above the top.
45	WCAG source scan (all pages)	An accessibility scan of 109 pages at up to three sizes, with the legend expanded so hidden text is measured too. Level AA of the current standard is blocking.

Do the promises match reality? (the honesty checks)

19	log retention == privacy.html	Reads the retention promise off the privacy page, asks Amazon what it is actually doing, and fails on any difference.
53	aws perms == iam-policy.json	<i>Exercises 18 safe Amazon actions to see which the deploy pass really has. Found a total outage on its first run: a console edit had replaced the whole permission list. "A permission is a timestamp, not a property."</i>
54	deployed == source	<i>Downloads 134 files from the live site and compares each with the working copy, so a forgotten upload is named.</i>
55	site == /v1/score (6 postcodes)	<i>The score the live website draws against the live API, for six postcodes.</i>
56	responsive, live	<i>Check 44, run against the live site instead of the working copy.</i>
57	live web == tabbed layout	<i>The live phone layout is the one that was deployed on purpose.</i>

Housekeeping (advisory)

46	Prettier (all files deviate)	<i>Code formatting. Deliberately advisory: it would flag every file, and a permanently red check is an ignored check.</i>
47	npm audit	<i>Known vulnerabilities in the third-party packages the tooling uses.</i>

3 - The rule that makes the checks worth having

A check that has never gone red has not been tested. Before a new check is trusted, the thing it guards is broken on purpose, the check is watched turning red, the break is undone, and the check is watched turning green. Only then does it count.

The rule exists because twelve checks in this project's history were green for months while checking nothing. Nobody switched them off; each decayed one sensible decision at a time:

- The whole checklist once piped every result through a command that always succeeds, so it said PASS while running nothing.
- The accessibility scan covered one page in nine and only the most severe faults; every real fault was one grade lower.
- A minimum of "at least 17 files" was left in place the day after the 18th file was added, so it could no longer notice that one going missing.
- A test took its expected answers from the very list it was testing, so the two could never disagree.
- A test file that guarded "the fallback shipped untested" was itself in no runner at all.

The question that finds these is not "what does this check run?" but "**what would this check miss?**". Names are treated as claims: a check called "all pages" that scans one page is the warning sign.

4 - How the coding assistant reaches Amazon

Most of the code was written with Claude Code, a coding assistant that reads and writes the project's files under direction. It runs on my computer, not on Amazon. When it needs to touch the live system, here is the path, and where it is narrow.

1 Me, at the keyboard

I give the instruction. Nothing below happens without one, and going live needs an explicit "go".



2 Claude Code, on my laptop

Reads and edits files, runs the checks, runs commands in the terminal. It has no account of its own with Amazon.



runs a command such as "upload this file" or "deploy the blueprint"

3 Two guards on every command

A screening hook blocks destructive commands outright (delete, force-overwrite, wipe). Claude Code's own safety classifier refuses commands it judges to be a production deploy unless I run them myself. Both have fired for real; see Part 7.



4 Amazon's command-line tools, using a named pass

The command runs Amazon's own tools (the AWS CLI, and SAM for the blueprint) with a pass called "flightmap" that lives in a file on my laptop, outside the project, never in the code.



over the internet, to Amazon's London region

5 Amazon checks the pass against its written permission list

The pass belongs to one identity ("flightmap-dev") with one written list of what it may do. Every command is checked against that list before anything happens.



6 Only Sky Score's own things

Most of the list is restricted to things named "london-flight-map-*": the eight workers, the five card boxes, the one blueprint, the one website bucket. Two parts are wider than that, and are described honestly below.

What the pass cannot do, and where it can do more than it should

- **Edit its own list directly.** When a new permission is needed, I paste it in through Amazon's web console myself, which needs my password and my phone. **But this is not yet a hard wall, and I would rather say so.** To deploy the workers, the pass may create worker roles and attach permissions to them, and nothing yet limits *which* permissions. A determined holder of the pass could use that to give a new role far more than Sky Score needs. Closing it is written down as an open item (a "permissions boundary"); it waits for a console session because an untested change to these rules caused a real outage once already.
- **A second wide grant.** The part of the list that manages the web front door (API Gateway) covers every such front door in the London region, not only Sky Score's. There is only one today, but the list does not enforce that.
- **Use the web console at all.** The pass is for command-line tools only. The console is mine, protected by a second factor on my phone.
- **See the bill, other projects, or other people's accounts.** Not on the list, though the gap in the first point is a way round that until it is closed. Other people's accounts are out of reach either way.
- **Empty a card box or delete the website bucket.** The screening hook blocks delete commands before they reach Amazon, and the list keeps deletions to Sky Score's own named things even if one got through.

Where the secrets live

- The Amazon pass: a file in my home folder, outside the project. Never copied into the code.
- The API keys and the government energy-data token: a file called `.env` in the project folder that the version history is told to ignore. The energy token is handed to the blueprint at deploy time as a hidden value, so it never appears in Amazon's own records of the deploy.

- If a secret ever appears in a chat log or a terminal, it is treated as burned and replaced. There is a script for replacing the energy token that refuses to run with the old value.
- **The automated tests that run on GitHub have no Amazon pass at all.** They lint and test; they cannot deploy. Nothing reaches the live system except from my laptop, on my instruction.

5 - Five layers of guardrail

- 1 **The 57 checks.** Nothing is saved into the project until they pass, and a saved change is the only kind that gets deployed.
- 2 **The screening hook.** Every terminal command the assistant proposes is read first; destructive ones are blocked before they run.
- 3 **The safety classifier.** The assistant's own judgement of "this looks like a production deploy", which makes it stop and hand the command to me.
- 4 **The written permission list on Amazon's side.** Even a command that gets past everything above can only touch Sky Score's own named things.
- 5 **My go-ahead.** The working rule: the assistant deploys nothing without an explicit instruction, and a deploy is verified afterwards from the live address, never from the command's own "success" message.

The layers are deliberately different in kind: a checklist, a filter, a judgement, a permission, a person. A single guardrail, however good, has one way to fail.

6 - What a deploy looks like, step by step

- 1 **Preflight passes** (Part 1) and the change is saved into the project's history.
- 2 **Back end first.** SAM reads the blueprint, works out what would change ("change set"), and applies it. Only the workers whose code changed are replaced. If a data edition rolled, the API must go live before the website, so the site's copy of the numbers is never ahead of the API's.
- 3 **Then the website**, from a deploy file with one named target per surface (fonts, pages, data, app files, demo, the 100 borough pages, and now the talk write-ups). The order inside it matters: data files before the page that fetches them, fonts before the offline worker that pre-caches them.
- 4 **Then the caches are cleared** at the copies-in-every-city, so visitors get the new version and not last hour's.

- 5 Then verify from the origin.** Download the live files and compare with what was written (check 54). Ask the live API for the 28 postcodes (check 8). A deploy is done when the live address says so, not when the upload command said "complete". On 26 August every upload succeeded and the cache-clear failed at the very end; the deploy looked done and was not.

7 - Two days the guardrails said no

16 September. A data release morning: the Land Registry's July prices, a methodology change, two defects found by the checks. When it came to the back-end deploy, the assistant's safety classifier refused to run it from its own session. I ran the one deploy command myself. That is the arrangement working as designed: the assistant prepared and verified everything up to the line, and the line held.

21 September, the day this was written. After uploading the plain-English write-up to the site, the assistant tried to fetch the live address to compare the file's fingerprint with the original. The classifier refused that too, judging the fetch to be part of the deploy. So the assistant stopped, said what it had tried to do and why, and asked me to run the one-line check. Thirty seconds of my time, and the rule stayed a rule rather than becoming a rule with exceptions.

What the second day shows. A guardrail that only ever blocks the dangerous thing is easy to keep. One that sometimes blocks a harmless thing is the kind you are tempted to loosen. Keeping it costs a little friction on ordinary days; loosening it costs the one day it mattered.

Words used above

Preflight

The 57-check list that runs before a change is saved. Named after the checks pilots run before take-off.

Blocking / advisory

A blocking check can refuse the change; an advisory one only warns.

Red / green

A check failing / passing. "Prove it goes red" means break the thing on purpose and watch the check fail before trusting it.

Pass (AWS)

A pair of long codes that identify one Amazon identity to Amazon's tools. Kept in a file, never in the code.

IAM

Amazon's system of identities and written permission lists. Every command is checked against the caller's list.

Blueprint / SAM

The one file describing every Amazon piece Sky Score uses, and the tool that makes the real thing match it.

Change set

SAM's list of what a deploy would alter, produced before it alters anything.

Deploy

Put a new version live.

Classifier

The coding assistant's own judgement about whether a command is safe to run without a person. It errs on the side of asking.

Sky Score · stage names from the preflight report of 2026-09-21 (45 PASS, 12 advisory, 23 min 34 s); permission facts from the deploy identity's policy file and the CI workflow; the two refusals from the sessions of 16 and 21 September 2026.